

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4002 Natural Language Processing
Group Project

Name	Matriculation Number
Dohyun Jeong	U2123042D
Goh Jie Rong, Sean	U2322220J
Tan Kuan Kiat	U2221895K
Chua Shun Kah	U2222915D
Rana Khushi	U2323605C

Name	Contribution
Dohyun Jeong	Part 1
Tan Kuan Kiat Chua Shun Kah	Part 2
Goh Jie Rong, Sean Rana Khushi	Part 3
Dohyun Jeong Tan Kuan Kiat Chua Shun Kah Goh Jie Rong, Sean Rana Khushi	Report

1. Part 1. Preparing Word Embeddings

In this part, we prepare the word embeddings for the tasks in the later part. We are asked to find the vocabulary size and OOV(out-of-vocabulary) words in the training data using a pre-trained word vector dictionary.

First, we used a tokenization method built in torchtext with `TEXT.build_vocab` function with GloVe pre-trained vectors, specifically `"glove.6B.100d"`, which contains 400,000 vocabs pretrained from 2014 Wikipedia (Pennington et al., 2025) with minimum frequency of 3. Their labels were also extracted with `LABEL.build_vocab`. After the tokenization, we can find the vocabulary size by finding the number of items in `TEXT.vocab`, which includes <unk> and <pad> tokens.

To obtain the OOV words, we first created a list of all of the words not in the vocabulary, then created a unique list to find the total number of OOV words. We also utilised the labels given to each word to find OOV words by topic.

While experimenting, we have found out that removing the minimum frequency makes the number of OOV words to be zero, so for subsequent parts we have decided to use tokenization with no minimum frequency.

Finally, we have extracted top 20 words by topic, and used t-SNE to convert obtained embeddings for top 20 words with GloVe to project on 2D space.

(a) What is the size of the vocabulary formed from your training data according to your tokenization method?

Initially, with the minimum frequency of 3, the size of the vocabulary was **2265**, including <unk> and <pad>. After the mitigation of OOV words done in part 1c, the size of the vocabulary now becomes **9330**.

(b) We use OOV (out-of-vocabulary) to refer to those words appeared in the training data but not in the Word2vec (or Glove) dictionary. How many OOV words exist in your training data? What is the number of OOV words for each topic category?

There were **7065** OOV words in the training data, using GloVe.6B for embedding. The table on the right contains the number of OOV words by topic.

Topic	OOV words
DESC	1613
ENTY	2035
ABBR	80
NUM	2062
HUM	1046
LOC	1060

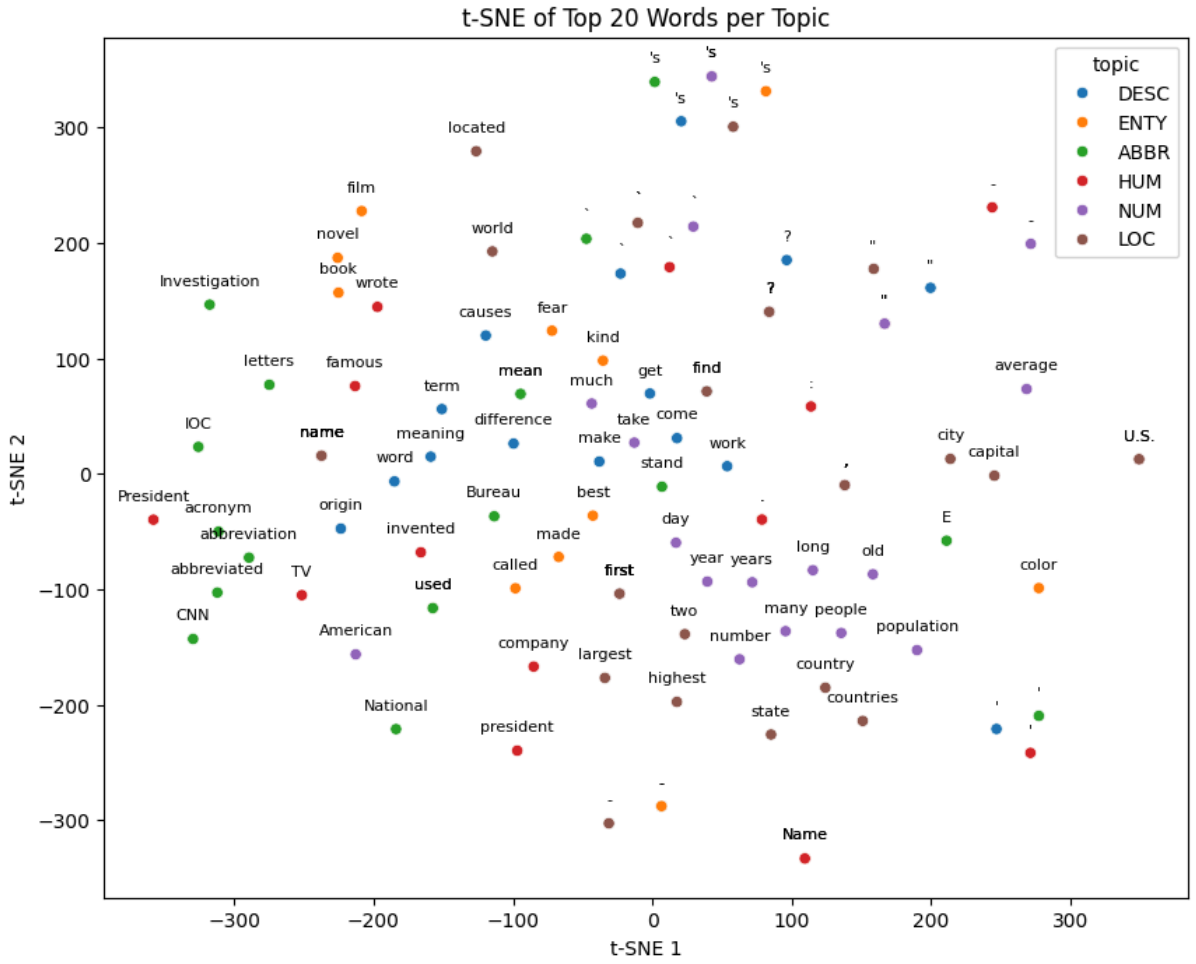
(c) The existence of the OOV words is one of the well-known limitations of Word2vec (or Glove). Without using any transformer-based language models (e.g., BERT, GPT, T5), what do you think is the best strategy to mitigate such limitation? Implement your solution in your source code. Show the corresponding code snippet.

There are several strategies that can mitigate the limitation of OOV words. One way to handle it is to use <unk> token, or simply ignore the OOV words. This method has a limitation of not capturing the semantics of OOV words. Some other strategies could include subword tokenization, which is to divide words to smaller subwords. However, some OOV words such as ‘wide’ and ‘2001’ would not work. We could use character level tokenization or FastText’s N-gram approach to ensure that no OOV is present, but in our case, simply removing the minimum frequency of 3 removed all OOV words. After the removal of minimum frequency, the size of the vocabulary was 9330.

Below is the code snippet that redo the tokenization with no minimum frequency.

```
TEXT.build_vocab(train_data,
    max_size=25000,
    vectors="glove.6B.100d",
    unk_init=torch.Tensor.normal_)
LABEL.build_vocab(train_data)
```

(d) Select the 20 most frequent words from each topic category in the training set (removing stopwords if necessary). Retrieve their pretrained embeddings (from Word2Vec or GloVe). Project these embeddings into 2D space (using e.g., t-SNE or Principal Component Analysis). Plot the points in a scatter plot, color-coded by their topic category. Attach your plot here. Analyze your findings.



Punctuations like "?", "", "`" and "-" are grouped with itself. Additionally, common words across different topics, such as “name”, “first”, and “find” are either located close to each other or make an overlap. Similar words like “country” and “countries” or “year” and “years” are also located close to each other. However, the presence of uppercase letter makes some words lie on completely different parts of the plane, such as “president” around (-100, -250) while “President” around (-350, -50), and “name” around (-200, 0), “Name” around (100, -300).

Generally, most words within the same topic have similar values of t-SNE, resulting in the words lying around the same region on the 2D graph such as words within the NUM topic located around (-100, 100). This shows that words with the same topic will generally have similar values of embedding vectors, while punctuations will have similar embedding vectors across all the topics.

2. Part 2. Model Training and Evaluation - RNN

This section covers the implementation,training and evaluation of a Recurrent Neural Network (RNN) model for topic classification using the preprocessed dataset from Part 0 as well as the pretrained word embeddings from Part 1. The pretrained embeddings are used as the embedding layer and fine-tuned during training to improve specific task performances.

To mitigate the effects of out-of-vocabulary (OOV) words, unknown tokens <unk> embeddings are initialised with the mean of all pretrained vectors while the padding token embedding is set to zero.

The RNN model architecture consists of an embedding layer, recurrent layer and a fully connected linear output layer. The recurrent layer is configured as a single layer bidirectional LSTM. The output from the final states of both directions are concatenated to form the sentence representative which is then passed for topic prediction.

The model is trained using the Adam optimizer and cross-entropy loss is used as the objective function. Training progress is monitored through both training and validation accuracy across epochs where the model achieving the highest validation accuracy is saved for final evaluation.

Upon completion of training, the best performing model is evaluated on the test dataset. Performance accuracy is then complemented by a confusion matrix to analyze the per-topic accuracy and misclassification patterns.

(a) Report the final configuration of your best model, namely the number of training epochs, learning rate, optimizer, batch size and hidden dimension.

After conducting hyperparameter tuning, the number of training epochs is **15**, learning rate is **0.001**, optimizer used is Adam, batch size used is **64**, dropout rate is **0.5**, number of layers is **1** and the number of hidden dimensions is **128**.

(b) Report all the regularization strategies you have tried. Compare the accuracy on the test set among all strategies and the one without any regularization.

Regularization techniques used include dropout, gradient clipping and early stopping. Dropout rate used is **0.5**, gradient clipping is set to **1.0** and the patience for early stopping is **5**.

Strategy	No regularisation	Dropout only	Gradient Clipping only	Early Stopping only	All combined
Accuracy(%)	85.10	85.68	85.97	84.42	87.38

The combined effects of the different regularisation techniques addresses different problems, where dropout and early stopping prevents overfitting, gradient clipping prevents exploding gradients. The highest accuracy amongst all the regularisation strategies is where all the regulisation strategies are combined at **87.38%**. Compared to without regularisation, it shows a **2.28%** increase, showing that proper regularisation significantly helps generalisation.

Gradient clipping alone is the second best at **85.97%**, a **0.87%** increase from the one without any regularisation. This could be because the original RNN was suffering from gradient instability such as exploding gradients, and since gradient clipping alone has a better improvement than dropout

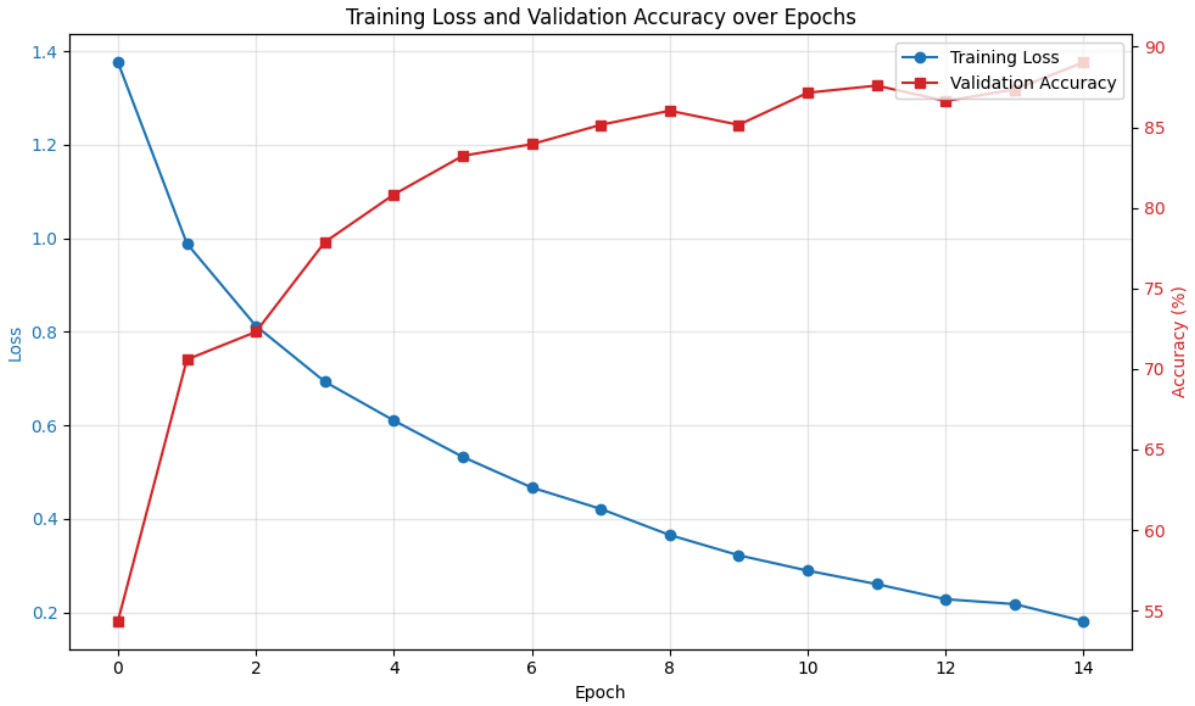
alone, it could mean that training stability serves to be a bigger issue than overfitting for the RNN model.

Dropout alone has a smaller increase of **0.58%**, which could indicate that mild overfitting was present but it was not the primary limiting factor. Dropout can sometimes make gradient flow worse without gradient clipping as it can cause underfitting and disrupt normalisation statistics in batch normalisation.

Early stopping alone performed worst with a **0.68%** drop. With a patience of 5, the model had likely stopped training before it fully converged, and since the model was not overfitting as much, early stopping might have prevented it from reaching its optimal performance. This suggests that the model might have needed more training time.

The combined regularisation strategy works with early stopping because with dropout and gradient clipping, the training is more stable with lesser validation fluctuation and early stopping can work properly after true convergence and not false plateaus because the improvements are now more consistent.

(c) For the best configuration and regularization strategy in your experiments, plot the training loss curve and validation accuracy curve during training with x-axis being the number of training epochs. Discuss what the curves inform about the training dynamics.



The training loss decreases rapidly in the first 3 epochs, dropping from 1.4 to 0.8. After epoch 3, the loss continues to decrease but at a much slower rate. The training loss is smooth without any erratic fluctuations and by epoch 12 to 14, the training loss has stabilised around 0.2.

The validation accuracy increases sharply in the first 3 epochs, rising from 40% to 75%. Between epoch 3 to 8, the accuracy continues to improve but at a much slower rate. There are some minor fluctuations but the general trend is increasing. The validation accuracy hovers around 85% around epoch 12 to 14.

The steep improvement in the first few epochs shows that the model quickly learns the most important features. The convergence at epoch 10 to 12 suggests that the learning rate is suitable and the model capacity is sufficient for the task.

The smooth curves indicate a stable gradient flow from gradient clipping, the lack of the vanishing gradient problems, effective batch normalisation from dropout and the batch size is appropriate for stable updates.

(d) RNNs produce a hidden vector for each word, instead of the entire sentence. Which methods have you tried in deriving the final sentence representation to perform sentiment classification? Describe all the strategies you have implemented, together with their accuracy scores on the test set.

The hidden state from the final time step of the RNN is used where it concatenates forward and backward final states. The model extracts the last hidden states from the final RNN layer:

```
if self.bidirectional:
    # take forward and backward for the last layer and c
    forward_h = h_n[last_layer_index, :, :]
    backward_h = h_n[last_layer_index + 1, :, :]
    h_final = torch.cat((forward_h, backward_h), dim=1)
else:
    h_final = h_n[last_layer_index, :, :]
```

This vector is then passed through a dropout and a fully connected layer for classification.

```
h_final = self.dropout(h_final)
logits = self.fc(h_final)    # [batch, output_dim]
return logits
```

The last hidden state helps summarize the information from the sequence, especially effective for short or medium-length sentences.

The bidirectional RNN can capture both the question words and contextual clues which helps to capture the entire sentences instead of singular words. Final accuracy for this strategy is **87.38%**.

(e) Report topic-wise accuracy (accuracy for each topic) on the test set for the best model you have. Discuss what may cause the difference in accuracies across different topic categories.

Topics	DESC	HUM	LOC	NUM	ENTY	ABBR
Accuracy(%)	98.55	93.85	92.59	91.15	82.98	66.67

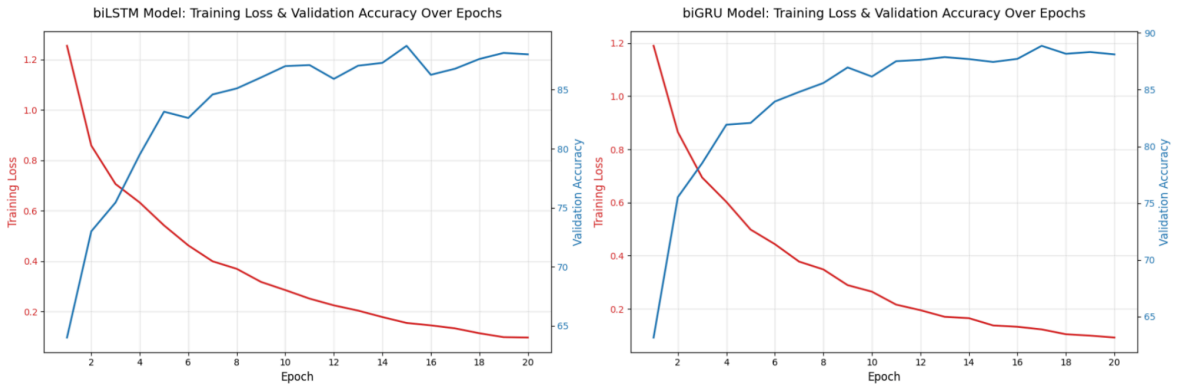
DESC has the highest accuracy with **98.55%** because of their strong linguistic patterns and clear intent markers and have little ambiguity with other categories. **HUM** is second best with **93.85%** accuracy because it also gives clear entity indicators and starts out with distinct questions words. But may perform worse when the model confuses contextual clues. **LOC** also has a relatively good accuracy at **92.59%** due to clear question words and geographical markers. **NUM** has a lower accuracy at **91.15%** due to the diverse numeric subcategories and common confusion of question words with other categories. **ENTY** has a sharp drop in accuracy at **82.98%** due its broad and

vague definitions and its heavy overlap with other categories. **ABBR** has the lowest accuracy at **66.67%** due to its limiting training data which can cause severe underfitting.

3. Part 3. Enhancement

(a) Plot the training loss curve and validation accuracy curve of biLSTM and biGRU. Report their accuracies on the test set (Part 3.1).

Below are the training loss and validation accuracy curves of biLSTM and biGRU, as well as the screenshots of the console stating their accuracy on the test set.



The final test accuracies for the models are as follows:

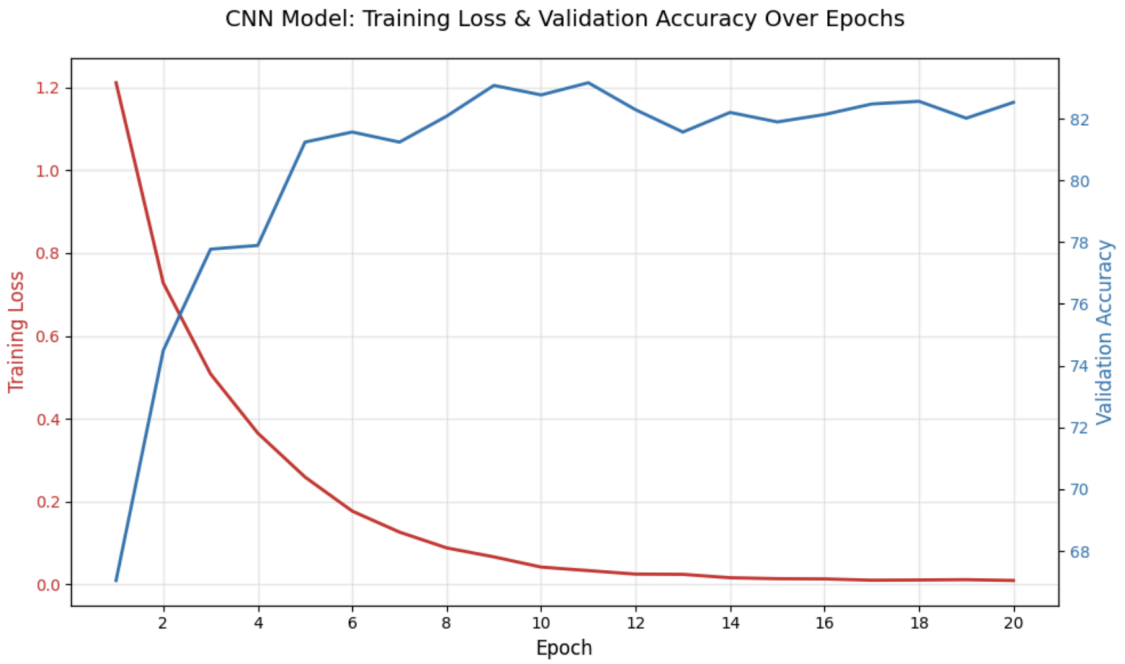
```
================================================================================
---- Loading best biLSTM model for final test evaluation ----

Final biLSTM Test Loss: 0.257 | Final biLSTM Test Accuracy: 92.80%
Final biLSTM Test Accuracy: 92.80%
================================================================================
================================================================================
---- Loading best biGRU model for final test evaluation ----

Final biGRU Test Loss: 0.205 | Final biGRU Test Accuracy: 95.18%
Final biGRU Test Accuracy: 95.18%
================================================================================
```

(b) Plot the training loss curve and validation accuracy curve of CNN. Report its accuracy score on the test set (Part 3.2).

Below are the training loss and validation accuracy curve of CNN, as well as the screenshots of the console stating their accuracy on the test set.




```
================================================================================
---- Loading best CNN model for final test evaluation ----

Final CNN Test Loss: 0.371 | Final CNN Test Accuracy: 88.07%
Final CNN Test Accuracy: 88.07%
================================================================================
```

- Final CNN Test Accuracy: 88.07%

(c) Describe your final improvement strategy in Part 3.3. Plot the corresponding training loss curve and validation accuracy curve. Report the accuracy on the test set.

As our final improvement strategy, we adopted a BERT-based classifier utilizing the pre-trained `bert-base-uncased` model. This approach fundamentally differs from our previous implementations that relied on static GloVe embeddings. BERT produces contextualized word representations where each word's vector is dynamically computed based on its surrounding context, enabled through bidirectional pre-training using masked language modeling and next sentence prediction (Devlin et al., 2019).

We anticipated that this contextual approach would provide more nuanced semantic understanding and improved handling of ambiguous terms. BERT offers several key advantages over our RNN-based models. Its WordPiece tokenization strategy effectively manages out-of-vocabulary words by decomposing them into subword units, addressing a major limitation we faced with fixed-vocabulary embeddings. Furthermore, BERT's bidirectional attention mechanism processes entire sequences simultaneously while considering both left and right context, contrasting with bidirectional RNNs that process sequences in two separate sequential passes. Additionally, the model benefits from pre-training on large text corpora, providing strong foundational linguistic understanding that transfers effectively to our topic classification task.



As shown in the image, the BERT model achieved a test accuracy of **96.8%**, which represents a significant improvement over the biGRU model (95.18%), biLSTM (92.80%), and even the CNN model (88.07%). The performance gain validated our hypothesis that contextualized embeddings and transfer learning from pretrained language models provide substantial benefits for sentence classification tasks.

```
================================================================================
---- Loading best BERT model for final test evaluation ----

Final BERT Test Loss: 0.160 | Final BERT Test Accuracy: 96.80%
================================================================================
```

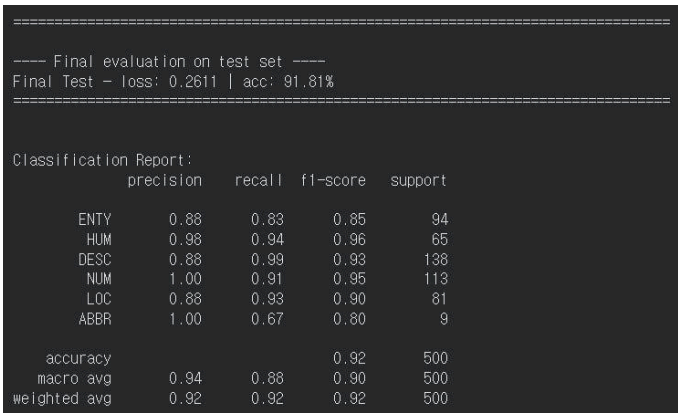
- Final BERT Test Accuracy: 96.8%

(d) Describe your strategy for improvement of weak topics in Part 3.4.
Report the topic-wise accuracy on the test set after applying the strategy and compare with the results in Part 2(e).

After analyzing the topic-wise accuracy from Part 2(e), we identified significant performance disparities across topic categories. Our baseline RNN model achieved an overall test accuracy of **87.38%**, but topic-wise analysis revealed substantial imbalances.

To address this issue, we implemented a **class weighting strategy** in our biGRU model. We used scikit-learn's `compute_class_weight` function with the 'balanced' mode, which automatically calculates weights inversely proportional to class frequencies. These weights were then applied to PyTorch's `CrossEntropyLoss` function, ensuring that misclassification of underrepresented or poorly-performing topics incurs higher penalties during training.

Analysis of Results



```
==========================================================================

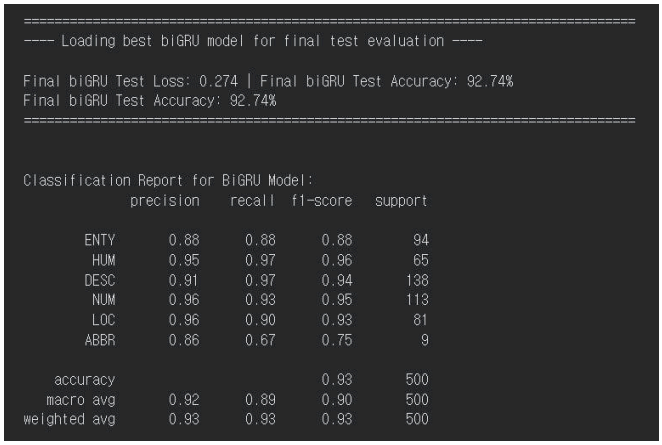
---- Final evaluation on test set ----
Final Test - loss: 0.2611 | acc: 91.81%
==========================================================================


Classification Report:
              precision    recall  f1-score   support

        ENTY       0.88      0.83      0.85        94
         HUM       0.98      0.94      0.96        65
        DESC       0.88      0.99      0.93       138
         NUM       1.00      0.91      0.95       113
         LOC       0.88      0.93      0.90        81
        ABBR       1.00      0.67      0.80         9

    accuracy                           0.92       500
   macro avg       0.94      0.88      0.90       500
weighted avg       0.92      0.92      0.92       500
```

Before applying class weighting, our RNN model showed significant performance disparities across topic categories. **ENTY demonstrated the poorest recall at 83%**, indicating the model frequently failed to detect entity-type questions, often confusing them with DESC and other categories. **ABBR showed critically low recall at 67%** with only 6/9 samples correctly identified, though it achieved perfect precision (1.00), revealing the model's extremely conservative prediction behavior due to severe class imbalance. The strong-performing category **DESC achieved 99% recall** (137/138 correct), highlighting a **performance gap of 32 percentage points** between the best (DESC: 99%) and worst (ABBR: 67%) performing topics.



```
==========================================================================
---- Loading best biGRU model for final test evaluation ----

Final biGRU Test Loss: 0.274 | Final biGRU Test Accuracy: 92.74%
Final biGRU Test Accuracy: 92.74%
==========================================================================


Classification Report for BiGRU Model:
              precision    recall  f1-score   support

        ENTY       0.88      0.88      0.88        94
         HUM       0.95      0.97      0.96        65
        DESC       0.91      0.97      0.94       138
         NUM       0.96      0.93      0.95       113
         LOC       0.96      0.90      0.93        81
        ABBR       0.86      0.67      0.75         9

    accuracy                           0.93       500
   macro avg       0.92      0.89      0.90       500
weighted avg       0.93      0.93      0.93       500
```

After applying class weighting, the strategy produced mixed but overall positive results, with the model achieving an improved test accuracy of 92.74% (up from 91.81%, a +0.93 percentage point gain). The most notable success was ENTY, which showed significant improvement from 83% to 88% recall (F1-score: 85% to 88%), representing a +5 percentage point gain. This improvement indicates that the increased class weight successfully helped the model learn stronger decision boundaries to distinguish entity-type questions from DESC and other confusing categories. HUM also benefited from the rebalanced loss function, improving from 94% to 97% recall (maintaining F1-score at 96%), demonstrating that the strategy enhanced predictions for human-related questions. Additionally, NUM showed modest gains, increasing from 91% to 93% recall while maintaining its F1-score at 95%.

However, **ABBR's performance revealed the strategy's limitations**, with recall remaining unchanged at **67%** despite having the highest class weight. The F1-score actually decreased from 80% to 75% as precision dropped from 100% to 86%, demonstrating that **class weighting cannot overcome extreme data scarcity** when only 9 samples exist. Some high-performing categories experienced minor trade-offs: **DESC's recall decreased from 99% to 97%** (F1 improved: 93% to 94%) and **LOC's recall dropped from 93% to 90%** (F1 improved: 90% to 93%), representing acceptable precision gains. These results show class weighting effectively addresses **moderate imbalance** (ENTY, HUM) but fails for **extreme imbalance** (ABBR).

```
Topic-wise Accuracy:
Topic           Accuracy     Correct/Total
--------------------------------------------
DESC                98.55%    136/138
HUM                 93.85%     61/65
LOC                 92.59%     75/81
NUM                 91.15%    103/113
ENTY                82.98%     78/94
ABBR                66.67%      6/9
```

Note: Topic-wise accuracies shown are from the baseline RNN evaluation. Minor variations (±1%) may exist between different evaluation runs due to computational precision.

References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*.

Pennington, J., Socher, R., & D. Manning, C. (2025, July). GLOVE: Global Vectors for Word Representation. https://nlp.stanford.edu/projects/glove/